TP Sheet 3 - Localization

Prepared by: Gonçalo Leão
With contributions by: Professor Luís Paulo Reis and Professor Armando Sousa

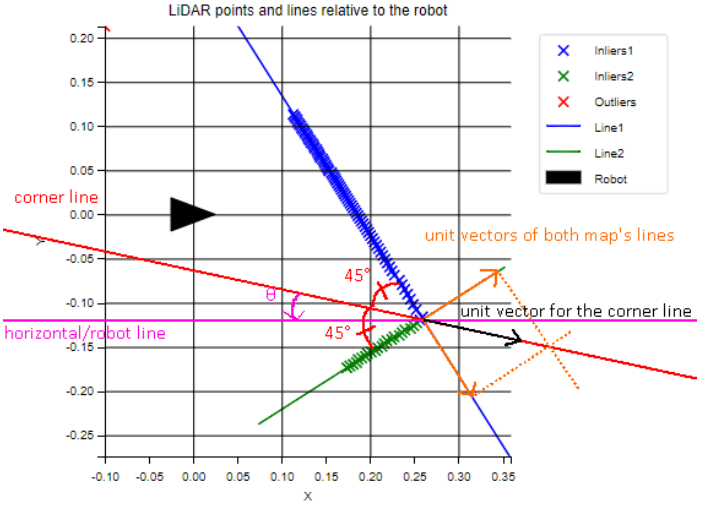
Exercise 1 - LiDAR-based localization using a corner and RANSAC

Consider the simulation scenario in the TP3_ex1.wbt Webots file, where the e-puck robot is facing a corner of a rectangular map and is equipped with a LiDAR with a field of view of 90 degrees.

Write a robot controller that returns the four possible estimated poses of the robot (position and orientation - x, y and angle) in the map, by locating the relative pose of the corner the robot is looking at, relative to the robot.

To localize the corner, use the Random Sample Consensus (RANSAC) algorithm twice to fit two lines to the point cloud captured by the LiDAR. The outliers of the first run of the RANSAC serve as the input for the second run. Each run of RANSAC returns the position of a point in the line and a unit direction vector.

The relative position of the corner is computed as the intersection between the two lines, while the relative angle (i.e. the angle that the robot needs to turn in order to face the corner at 45 degrees) can be computed using the scalar product between the unit vector of the robot's line of vision (i.e. a perfect horizontal line) and the line that makes 45º with each line of the corner, as shown below. The image shows an example of the exercise's plot to visualize the result of RANSAC.
Note: Make sure to flip the orientation of the map line's unit vectors so that they are facing the right (positive x component) and use math.atan2, so that the angle θ is computed with the correct sign.



With the relative position and orientation of the corner, the transformation matrix that aligns the robot frame with the corner frame (so that the robot is placed in the corner at an angle of 45º, facing outwards of the map), denoted as T_{corner}^{robot}, can be computed using the create_tf_matrix function already provided in transformations.py.

The four possible positions for the robot can be computed by combining T_{corner}^{robot} with the geometric transformations that align the map's origin with each corner, denoted as T_{corner}^{origin}. For instance, the geometric transformation for the upper right corner can be computed using a translation of (0.5, 0.5) and a rotation of +45 degrees (+pi/4 radians).

To help you, use the .py file with the base code provided in the Github repository and fill out these TODOs (in order):

1. In the find_corner_transformation function,
    a. Define the outliers of the result of the first RANSAC (suggestion: use a list comprehension where you use inliers).
    b. Complete the arguments for the second RANSAC call.
    c. Define the outliers of the result of the second RANSAC.
    d. Retrieve a list with the inliers of both calls to RANSAC.
2. In the line_line_angle function, perform the computation of the θ angle that was previously explained, between the robot's orientation and the bisector line of the corner.
3. In the find_possible_poses function,
    a. Define the transformations that align the map center's frame with the corners, T_{corner}^{origin}.
    b. Compute the possible translations and rotations of the robot in the map center' frame, by using get_translation and get_rotation (from transformations.py), on each T_{robot}^{origin}, computed by combining T_{corner}^{robot} and each T_{corner}^{robot}.
4. Test your solution in different scenarios:
    a. Try changing the robot's pose before running the controller.
    b. What happens if you decrease the number of rays?
    c. What if you increase the LiDAR's noise?

Exercise 2 - A more complete LiDAR-based localization using ICP

Initially, consider the simulation scenario in the obstacles.wbt Webots file. You can also experiment this exercise in other maps created using the create_map.py file, but you must first change the map_name variable (in the main function) to match the name of your map (and change the custom_maps_filepath if your Webots file is in another directory), so that the CSV file with the coordinates of the points of the correct map is retrieved.

Write a robot controller that returns an estimated pose of the robot in the map, by using the Iterative Closest Point (ICP) algorithm to estimate the transformation that aligns its LiDAR readings (moving point cloud) with the list of points corresponding to walls in the map (fixed point cloud).

Given that ICP requires a good initial estimate of the transformation to align the point clouds, a grid search will be used, where different matrices will be built using various values for the robot's initial (x,y) position and ψ rotation angle. For each matrix candidate ($T_{initial}$), ICP will be run and the error (mean square error is the nearest neighbors) is recorded. Then, ICP will be run with a larger number of iterations with the candidate that gave the smallest error. The initial transformation estimate ($T_{initial}$) and the result of this final ICP run ($T_{refined}$), are then combined to obtain the robot's actual transformation (T_{robot}^{origin}).

To test out the ICP algorithm, run the registration.py file in the ICP directory. This produces an example of the alignment of two 3D point clouds, including images of the step by step process and the evolution of the error per iteration.

To help you, use the .py file with the base code provided in the Github repository and fill out these TODOs (in order):

1. In the run_icp_with_inital_tf function, build the moving point cloud, by applying the initial transformation ($T_{initial}$) to each point of the LiDAR readings.
2. In the grid_search_icp,
    a. Define the grid search with different initial values values for the robot's position and orientation in order to find the initial pose / transformation ($T_{initial}$) with the smallest ICP error. For each candidate, you should call run_icp_with_inital_tf.
    b. Complete the arguments for the ICP call that produces the refined transformation ($T_{refined}$). You should use a smaller threshold for the maximum difference of error between iterations then on the call to ICP during grid search, so that more iterations are performed and a more accurate transformation is obtained.
    c. Compute T_{robot}^{origin} by combining $T_{initial}$ and $T_{refined}$.